

Chapter 1 — Databricks & Lakehouse Fundamentals

Topic 1.2 — Databricks Compute

This is an **important Professional-level topic** because many exam questions are scenario-based:

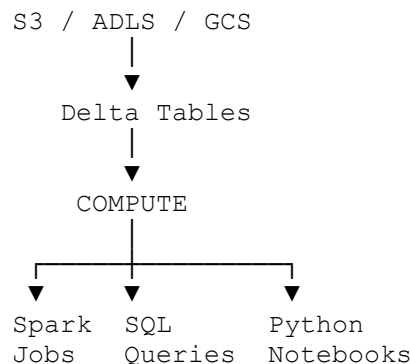
"Which compute should you choose for this workload?"

You need to understand **why**, not just memorize names.

1. What is Compute in Databricks?

Compute is the processing infrastructure used to execute your workloads.

Your data generally lives in cloud storage:

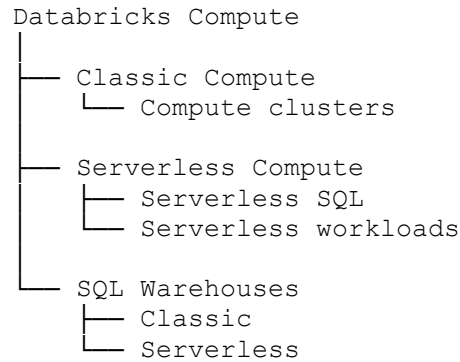


Compute performs operations such as:

- Reading data
 - Transforming data
 - Joining datasets
 - Aggregating
 - Running SQL
 - Running Python/PySpark
 - Running streaming pipelines
 - Executing jobs
-

2. Main Compute Types

For your exam, understand these major categories:

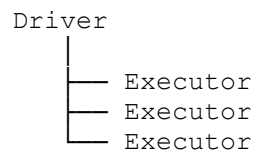


The exact available compute options can evolve, so on the exam always pay attention to the **workload and requirements**, rather than relying only on an old product name.

3. Classic Compute

Classic compute means you provision/manage compute resources such as a Spark cluster.

Conceptually:



The **driver** coordinates the Spark application.

The **executors** perform distributed processing.

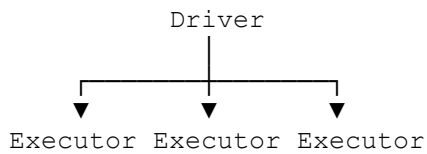
4. Driver

The Driver is essentially the **coordinator** of a Spark application.

It:

- Creates the Spark session/context
- Builds the execution plan
- Coordinates tasks
- Communicates with executors
- Tracks application progress

Conceptually:



Important

The driver does **not** normally perform all the distributed data processing itself.

Executors perform the distributed tasks.

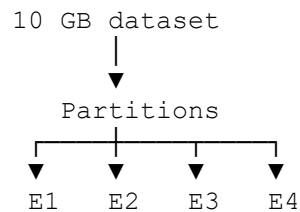
5. Executors

Executors are worker processes responsible for executing tasks.

They:

- Execute Spark tasks
- Process partitions
- Store cached data
- Perform transformations
- Exchange data during shuffles

Example:



Each executor can process one or more partitions.

6. Driver vs Executor

This is worth memorizing.

Driver	Executor
Coordinates application	Executes tasks
Creates execution plan	Processes partitions
Schedules tasks	Performs computation
Maintains application state	Can cache data
Communicates with executors	Sends results/status

Exam scenario

A Spark job fails because the driver runs out of memory while collecting a huge dataset.

The problem is likely **driver memory**, not executor memory.

For example:

```
df.collect()
```

can be dangerous when `df` is very large because the results are brought back to the driver.

7. Driver Memory vs Executor Memory

This is an important troubleshooting concept.

Driver memory problem

Potential causes:

```
df.collect()  
df.toPandas()
```

on a huge dataset.

Or:

- Huge query plan
- Too much driver-side state
- Excessive metadata

- Large broadcast-related operations

Executor memory problem

Potential causes:

- Large partitions
- Memory-heavy transformations
- Large joins
- Data skew
- Excessive caching
- Shuffle pressure

8. All-Purpose / Interactive Compute

Interactive compute is designed for workloads where users actively work with the environment.

Typical examples:

- Notebook development
- Exploration
- Debugging
- Interactive PySpark
- Data analysis

Conceptually:

```
Developer
  ↓
Notebook
  ↓
Interactive Compute
  ↓
Spark
```

This is useful during development.

9. Job Compute

For production workloads, you generally want compute designed around automated jobs rather than having engineers manually run notebooks.

Example:

```
Schedule
  ↓
Databricks Job
  ↓
Task
  ↓
Job Compute
  ↓
ETL
```

A job can:

- Start compute
- Execute tasks
- Retry failures
- Run dependencies
- Send notifications
- Terminate compute when finished

10. Why Job Compute Can Be Better for Production

Imagine an ETL pipeline runs every night at 2 AM.

Bad architecture:

```
Permanent interactive cluster
  ↓
Runs every night
```

The cluster may remain available even when nobody is using it.

Better:

```
2 AM
  ↓
Job starts
  ↓
Compute starts
  ↓
ETL executes
  ↓
Job completes
  ↓
```

Compute terminates

This can reduce unnecessary compute usage.

11. SQL Warehouses

A **SQL Warehouse** is designed primarily for SQL workloads.

Typical users:

- Data analysts
- BI developers
- SQL developers
- Reporting tools

Example:

```
Power BI
  ↓
SQL Warehouse
  ↓
Delta Tables
```

You can execute SQL such as:

```
SELECT
    region,
    SUM(revenue)
FROM gold.sales
GROUP BY region;
```

12. SQL Warehouse vs Spark Compute

A common exam decision.

Spark Compute

Best suited for:

- PySpark
- Python
- Complex data engineering
- Spark APIs

- General distributed processing
- Notebook workloads
- Some streaming workloads

SQL Warehouse

Best suited for:

- SQL analytics
- BI
- Dashboards
- SQL queries
- Business users

Think:

```
PySpark / Python
  ↓
Spark Compute

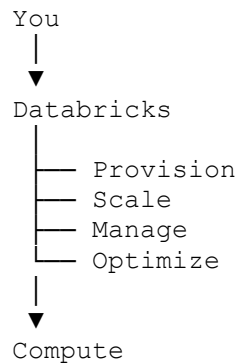
SQL / BI
  ↓
SQL Warehouse
```

13. Serverless Compute

Serverless means Databricks manages much of the underlying infrastructure for you.

You don't need to manually provision the underlying virtual machines in the same way as traditional compute.

Conceptually:



Benefits

- Faster startup
 - Less infrastructure management
 - Automatic scaling
 - Simplified operations
 - Potentially better developer experience
-

14. Serverless SQL Warehouse

A very common use case:

```
Power BI
  ↓
Serverless SQL Warehouse
  ↓
Unity Catalog
  ↓
Delta Tables
  ↓
Cloud Storage
```

This is especially useful for analytics/BI workloads where you don't want to manually manage clusters.

15. Photon

💡 **Important exam topic.**

Photon is Databricks' native vectorized query engine designed to accelerate SQL and DataFrame workloads.

Think:

```
Traditional Spark execution
  ↓
Spark Engine

Photon-enabled execution
  ↓
Photon
```

Photon is designed to improve performance for supported workloads.

16. Why Photon Can Be Faster

Photon uses techniques such as:

- Vectorized processing
- Native execution
- Efficient CPU utilization
- Optimized execution of supported operations

Instead of processing data one record at a time conceptually:

Row 1
Row 2
Row 3
Row 4

vectorized processing works on batches of values.

Conceptually:

Batch of values



↓

Vectorized CPU processing

This can significantly improve analytical workloads.

17. Important Photon Exam Trap

Don't assume:

"Photon makes every Spark/Python operation faster."

✗ Not necessarily.

Photon is primarily designed to accelerate **supported SQL and DataFrame workloads**.

If you're running arbitrary Python code that cannot benefit from Photon execution, don't expect Photon to magically optimize the Python code itself.

18. Autoscaling

Autoscaling allows compute resources to adjust based on workload requirements.

For example:

```
Low workload
  ↓
2 workers

Heavy workload
  ↓
8 workers

Work decreases
  ↓
3 workers
```

Benefits:

- Better resource utilization
- Potential cost savings
- Ability to handle workload variation

19. Important Autoscaling Concept

Autoscaling is **not a replacement for good Spark optimization**.

If your job is slow because of:

- Data skew
- Bad joins
- Excessive shuffles
- Poor partitioning
- Huge single partition

simply increasing workers may not solve the underlying problem.

Example:

```
Bad join
  ↓
Huge shuffle
  ↓
Add 50 workers
```

↓
Still inefficient

You need to address the root cause.

20. Cluster Policies

Cluster policies allow administrators to control how compute resources can be configured.

For example, an organization might enforce:

```
Maximum workers = 20  
Allowed DBR versions = approved versions  
Instance types = approved types  
Auto termination = required
```

Why?

Governance

Prevent users from creating expensive or non-compliant infrastructure.

21. Auto Termination

Auto termination shuts down idle interactive compute after a configured period.

Example:

```
Developer stops using cluster  
↓  
Idle  
↓  
30 minutes  
↓  
Cluster terminates
```

This helps reduce unnecessary costs.

Exam scenario

Engineers frequently forget to terminate development clusters.

Possible solution:

Configure auto-termination.

22. DBR — Databricks Runtime

Databricks Runtime provides the software environment used to run workloads.

It includes components such as:

- Apache Spark
- Databricks optimizations
- Libraries
- Runtime components

Different workloads may use different runtime versions.

23. Runtime Version Selection

Don't blindly choose the newest runtime.

Production environments generally need:

- Tested runtime
- Supported runtime
- Compatible libraries
- Stable behavior

A Professional-level question may describe a production pipeline that suddenly breaks after a runtime upgrade.

Possible solution:

- Test runtime upgrades before production
 - Use controlled versions
 - Validate dependencies
 - Follow organization's approved runtime policy
-

24. Compute Access Modes

You should understand the purpose of different access/isolation modes rather than simply memorizing names.

The key idea is:

Access mode determines how workloads and users access data and how isolated the compute environment is.

This becomes particularly important with:

- Unity Catalog
- Multi-user environments
- Security
- Data access
- Governance

We'll cover Unity Catalog access modes in much greater depth later.

25. Choosing the Right Compute — Exam Framework

When you see a compute question, ask these questions **in order**:

Question 1

What workload?

SQL/BI → SQL Warehouse
PySpark → Spark Compute
Streaming → Streaming-capable compute
Automated ETL → Job workload

Question 2

Interactive or production?

Development → Interactive
Production → Job

Question 3

Who manages infrastructure?

Less management → Serverless
More control → Classic compute

Question 4

Does the workload benefit from Photon?

For supported SQL/DataFrame workloads, Photon can be advantageous.

Question 5

Is workload size variable?

If yes, consider autoscaling.

26. Example Professional Scenario

Scenario

A company has a Power BI dashboard querying Delta tables.

The dashboard receives hundreds of concurrent SQL queries.

Which compute is the most appropriate?

Think

Power BI
↓
SQL workload
↓
Concurrent analytics

Answer

SQL Warehouse, with the appropriate scaling configuration.

Don't choose a general-purpose Spark cluster simply because the underlying data is Delta.

27. Scenario — Data Engineering

A data engineer needs to run:

```
df = spark.read.table("bronze.orders")

result = (
    df.filter(...)
      .join(...)
      .groupBy(...)
      .agg(...)
)

result.write.mode("append").saveAsTable("silver.orders")
```

This is primarily a **Spark/PySpark workload**.

Appropriate compute:

Spark-based compute / job compute, depending on whether this is development or production.

28. Scenario — Nightly ETL

Requirement:

Run ETL every night at 1 AM and automatically stop compute when finished.

Think:

```
Scheduled
+
Production
+
Automated
```

Use:

Databricks Job + job-oriented compute

rather than keeping an interactive development cluster running permanently.

29. Scenario — Development

Requirement:

Data engineer is developing and debugging a PySpark notebook throughout the day.

Think:

Interactive
+
Notebook
+
PySpark

Use:

Interactive compute.

30. Scenario — Cost Optimization

Requirement:

Developers frequently leave interactive clusters running overnight.

Possible solutions:

- Auto-termination
- Cluster policies
- Appropriate compute sizing
- Serverless where appropriate
- Job compute for automated workloads

Don't immediately conclude:

"Add more workers."

That's a **performance** solution, not necessarily a **cost** solution.

 **WRITE THIS IN YOUR
HANDWRITTEN NOTEBOOK**

Don't copy the whole chapter. Write this:

DATABRICKS COMPUTE

=====

Compute = processing infrastructure.

MAIN TYPES

1. Spark / Classic Compute
 - PySpark
 - Python
 - Spark workloads
 - Engineering
2. SQL Warehouse
 - SQL
 - BI
 - Dashboards
 - Analytics
3. Serverless
 - Databricks manages infrastructure
 - Less operational overhead
 - Automatic scaling
4. Job Compute
 - Production / automated workloads
 - Job starts → executes → terminates

SPARK ARCHITECTURE

Driver

- Coordinates application
- Creates execution plan
- Schedules tasks

Executors

- Execute tasks
- Process partitions
- Cache data
- Perform computation

DRIVER vs EXECUTOR

Driver OOM:

- collect()
- toPandas()
- Huge driver-side state

Executor problems:

- Large partitions
- Shuffle
- Skew

→ Memory-heavy operations

SQL WAREHOUSE

SQL + BI + dashboards

Spark Compute

PySpark + Python + Spark

PHOTON

Databricks native vectorized query engine.

- Accelerates supported SQL/DataFrame workloads
- Better CPU utilization
- Not automatically faster for arbitrary Python code

AUTOSCALING

Workers increase/decrease based on workload.

NOT a replacement for:

- fixing skew
- bad joins
- excessive shuffle
- poor partitioning

CLUSTER POLICY

Controls compute configuration.

Used for:

- Cost control
- Governance
- Security
- Standardization

AUTO TERMINATION

Stops idle interactive compute

- Reduces unnecessary cost

EXAM FRAMEWORK

SQL / BI → SQL Warehouse
PySpark → Spark Compute
Development → Interactive
Production ETL → Jobs
Less infrastructure management → Serverless
Variable workload → Autoscaling

10 Things You Must Be Able to Answer

Before moving ahead, make sure you can answer these without looking:

1. **What is the difference between a driver and executor?**
2. **Why can `collect()` cause driver OOM?**
3. **When would you choose SQL Warehouse over Spark compute?**
4. **When would you use interactive compute?**
5. **When would you use job-oriented compute?**
6. **What problem does serverless solve?**
7. **What is Photon?**
8. **Does adding more workers always solve a slow Spark job?**
9. **Why use cluster policies?**
10. **Why use auto-termination?**

If you can explain those **in your own words**, this topic is sufficiently covered for now.